

How To GreasyPidgin

Contents

1. Was ist GreasyPidgin?	1
2. Installation	3
2.1 Python installieren	3
2.2 Optional: Installation von miniconda	3
2.3 Installation von GreasyPidgin	3
3. Pitch	6
3.1 Aufbau	6
3.2 Umrechnungen	6
4. Envelope	9
4.1 Konstruktoren	9
4.2 Werte ausgeben	9
4.3 Envelope anzeigen	10
5. Grid	12
5.1 Ein Grid erzeugen	12
5.2 Quantisieren	14
5.3 Sieben	14
5.4 Sampling	17
6. TimeGrid	19
6.1 Vererbte Grid-Konstruktoren	19
6.2 Mikrotime-basierte Konstruktoren	20
6.3 Midi-Export	21
7. PitchGrid	23
7.1 Stimmungssysteme	23
7.2 Skalen	24
7.3 Konstruktoren	25
7.4 Export	26
8. DynamicGrid	29
9. TemporalObject	31
9.1 Was ist ein TemporalObject?	31
9.2 Ein TemporalObject erzeugen	31
9.3 Ein TemporalObject exportieren	32
10. Phonon	35
10.1 Was ist ein Phonon?	35
10.2 Ein Phonon erzeugen	35
10.3 Export	37

1. Was ist GreasyPidgin?

GreasyPidgin ist algorithmische Kompositionsumgebung, die versucht möglichst keine Vorgaben zu präsentieren, sondern mittels modularem Aufbau verschiedene Arten einer kreativen Anwendung zu ermöglichen. Nichtsdestotrotz gibt es eine kleine Anzahl von Festlegungen. GreasyPidgin basiert in erster Linie auf drei fundamentalen Systemen:

TemporalObject

Grid

Envelope

Hinzu kommen Erweiterungen, die sich an Konventionen traditioneller Komposition orientieren, wie

Ensemble

Player

Instrument

Pitch

GreasyPidgin beinhaltet außerdem eine Reihe algorithmischer Verfahren, die es erlauben Strukturen von Daten¹ zu erzeugen:

Lindenmayer-Systeme

Markov-Ketten

Zelluläre Automaten

Sieb- und Mengen- Operationen

Sowie dezidiert musikhistorisch algorithmische Verfahren:

Isorhythmie

Serielle Verfahren

¹...und somit auch klangliche bzw. musikalische Parameter

2. Installation

GreasyPidgin ist eine Umgebung, die vor allem in Python geschrieben ist, daher ist der erste Schritt sicherzustellen, dass Python installiert ist.

2.1 Python installieren

Unter <https://www.python.org/downloads> lässt sich die aktuelle Version von Python

2.2 Optional: Installation von miniconda

Python hat eine Tendenz, dass verschiedene Projekte verschiedene Versionen von Libraries benötigen, welche wiederum verschiedene Versionen von Python benötigen. Um sicherzustellen, dass bei der Arbeit mit GreasyPidgin keine Konflikte auftreten, empfiehlt es sich ein virtuelles Environment anzulegen, innerhalb dessen ausschließlich GreasyPidgin verwendet wird. Eine Möglichkeit dafür bietet Conda, zum Beispiel in der Variante miniconda

Unter <https://www.anaconda.com/docs/getting-started/miniconda/install> lässt sich die aktuelle Version von miniconda finden sowie Hinweise zur Installation. Um ein miniconda Environment für GreasyPidgin anzulegen, öffne ein Terminal und führe folgenden Code aus:

```
conda create --name greasypidgin python=3.14
```

um es zu aktivieren:

```
conda activate greasypidgin
```

2.3 Installation von GreasyPidgin

Lade das Repository von <https://gitea.com/sem-16161d/GreasyPidgin> wie folgt herunter. Klicke auf **Code** -> **Download ZIP** und entpacke die Datei in dem Verzeichnis, wo es landen soll. Öffne ein Terminal, und wechsle auf den Ordner, den du angelegt hast via `cd - change directory` – wie folgt:

```
cd path/to/directory/of/GreasyPidgin
```

Um GreasyPidgin zu installieren führe eine lokale Installation via `pip` aus. Stelle zuvor sicher, dass das GreasyPidgin Environment aktiv ist!

```
pip install -e .
```

Um sicherzustellen, dass GreasyPidgin installiert in dem Environment installiert ist, starte Python via

```
python3
```

Oder

```
python
```

Und führe dann in der Python-shell folgenden Code aus:

```
from GreasyPidgin.GreasyPidgin import GreasyPidgin
print(GreasyPidgin())
```

Im Terminalfester solltest du dann folgenden Printout erhalten:

```
#####
```

```
  .-.-.
  ( @° \
  < . . . \
  ) . . . \
  /-.-.-.-.-
  | GREASY  ( ) \
  \ PIDGIN  ( ) ( ) ( ) ; ) \ - - , _
```

[illegible]

```
name      : GreasyPidgin
id        : 0
start     : 0.000
dur       : 0.000
ensemble  : Ensemble(players=0, names=[],
tmp0bjs   : 0
```


3. Pitch

Eine Python-Adaption von Michael Edwards' `Pitch.lsp` aus seiner Kompositionsumgebung `slippery chicken`². `Pitch.py` enthält dabei keine eigenständige Klasse, sondern ist lediglich eine Sammlung von Funktionen. Das System basiert auf der *Scientific Pitch Notation (SPN)*³.

3.1 Aufbau

3.1.1 Oktavlagen

In diesem System ist der tiefste Ton als $c_0 = 16.3516 \text{ Hz}$ definiert, bei einem Kammerton von $a_4 = 440 \text{ Hz}$. Die Basis bilden die englischen Notennamen: A, B, C, D, E, F, G gefolgt von der Oktavlage von 0-9.

SPN	DE	MIDI	Hz ($a=440$)	Hz ($a = 442$)
c0	Subkontra C	12	16.35	16.42
c1	Kontra C	24	32.70	32.85
c2	großes C	36	65.4	65.70
c3	kleines c	48	130.81	131.4
c4	c'	60	261.626	262.81
c5	c''	72	523.25	525.63
c6	c'''	84	1046.5	1051.26
c7	c''''	96	2093	2102.52
c8	c'''''	108	4186	4205

3.1.2 Versetzungszeichen

Vorzeichen / Versetzungszeichen werden mittels Suffix indiziert: f für *flat* und s für *sharp*.

SPN	DE	MIDI	Hz ($a=440$)	Hz ($a = 442$)
af4	as'	68	415.3	417.19
cs3	kleines cis	49	138.59	139.22

Das gleiche gilt für Mikrotonalität⁴:

Suffix	EN	DE	CENT
qf	quarter tone flat	Viertelton tief	-50
qs	quarter tone sharp	Viertelton hoch	+50
sf	sixth tone flat	Sechstelton tief	-33
ss	sixth tone sharp	Sechstelton hoch	+33
tf	twelfth tone flat	Zwölftelton tief	-17
ts	twelfth tone sharp	Zwölftelton hoch	+17

3.2 Umrechnungen

Um zwischen den verschiedenen Tonhöhendarstellungen zu wechseln sind die Funktionen nach ihrer Entsprechungen in PureData und MAX/MSP benannt. Die Umrechnung Frequenz zu MIDI (*fom*) folgt der Formel⁵:

²<https://michael-edwards.org/sc/>

³wiki: [Scientific pitch notation](#)

⁴Edwards' Original nimmt EDO-72 als Standard an. Auch für den Gebrauch in einem Just-Intonation-System lassen sich die ersten 16 Partialtöne hinreichend genau mit diesem System beschreiben.

⁵Eine gute Herleitung und Zusammenfassung findet sich hier: <https://newt.phys.unsw.edu.au/jw/notes.html>

$$f(a_{ref}, MIDI) = a_{ref} * 2^{(MIDI-69)/12} \quad (3.1)$$

Und umgekehrt MIDI zu Frequenz (*mtof*):

$$MIDI(a_{ref}, f) = 69 + 12 * \log_2(f/a_{ref}) \quad (3.2)$$

Der default-Wert für den Referenz-Kammerton a_{ref} ist 440 Hz, lässt sich aber via *referenceA4* auf eine beliebige Frequenz ändern. *spntom* und *spntof* sind die Äquivalente um *SPN* in MIDI und Frequenz umzurechnen. Eine direkte Umwandlung von MIDI oder *Frequenz* in *SPN* ist zur Zeit noch nicht implementiert, da dieses die Bestimmung von Stimmungssystemen voraussetzt. Eine solche Umwandlung muss über *PitchGrid* erfolgen.

```
from GreasyPidgin.Pitch import mtof, ftom, spntom, spntof
pitch = 'c4'
midi = spntom(pitch)
freqDirect = spntof(pitch, referenceA4=440)
freqViaMtof = mtof(midi)
back2Midi = ftom(freqDirect)

print(
    " pitch          : ", pitch, '\n',
    "midi           : ", midi, '\n',
    "freq           : ", freqDirect, '\n',
    "freq(from Midi) : ", freqViaMtof, '\n',
    "freq to midi    : ", back2Midi, '\n',
)
```

OUTPUT

```
#####
#> pitch          : c4
#> midi           : 60.0
#> freq           : 261.6255653005986
#> freq(from Midi) : 261.6255653005986
#> freq to midi    : 60.0
```


4. Envelope

Jegliche Art von zweidimensionalen Abfolgen, Kurven und Hüllkurven wird in GreasyPidgin als Envelope dargestellt. Ein Envelope-Objekt kann entweder aus einer Reihe von Werten oder Wertepaaren – sogenannten *breakpoint pairs* – erzeugt werden. Während der Konstruktion werden die Werte innerhalb des Objekts normalisiert. Intern wird eine Envelope als Kurve in einem XY-Koordinatensystem gesehen.

4.1 Konstruktoren

4.1.1 Einzelwerte

Wird Envelope mit einem flachen list- oder tuple-Objekt initialisiert, werden die enthaltenen Werte in *breakpoint pairs* umgewandelt. Jedes Paar hat den gleichen x-Abstand zum benachbarten Paar.

```
from GreasyPidgin.Envelope import Envelope
```

```
env = Envelope([0,2,1,3,2,5,2,3,0,1,0])
print(env)
```

```
##### OUTPUT
```

```
#####
```

```
# Envelope
#   pointsNorm=[(0.0, 0.0), (0.1, 0.4), (0.2, 0.2), (0.3, 0.6), (0.4, 0.4),
#               (0.5, 1.0), (0.6, 0.4), (0.7, 0.6), (0.8, 0.0), (0.9, 0.2),
#               (1.0, 0.0)]
#   x_range=(0.0, 10.0), y_range=(0.0, 5.0)
#   coefficients={}
```

4.1.2 Wertepaare

Wird Envelope mit einem verschachtelten list oder tuple initialisiert, müssen die einzelnen Elemente **immer** x- und y-Wertpaare sein. In der Standardeinstellung werden die Elemente anhand ihres x-Wertes sortiert.

```
from GreasyPidgin.Envelope import Envelope
env = Envelope([(0,5),(1,1),(5,1),(4,2)])
print(env)
```

4.1.3 Polynom-Berechnung

Eine Möglichkeit Daten zu komprimieren ist die Berechnung von polynomischen Funktionen verschiedener Ordnung.⁶ Dies erlaubt zum Beispiel alternative Fortführungen der Kurve außerhalb des definierten Wertebereiches (siehe: *Werte ausgeben*)

4.2 Werte ausgeben

Mit der Methode `.getValue()` können Werte innerhalb der Kurve ausgegeben werden. Dabei gibt es sowohl die Möglichkeit x- als auch y-Werte normalisiert zu verwenden.

```
from GreasyPidgin.Envelope import Envelope
```

```
env = Envelope([0,1,2,3,2,1,0])
# get the y-value for the position 0.5 of 6
print("0.5 of 6: ",env.getValue(xPos=0.5, normalizedX=False))
# get the y-value for the position 0.5 of 1 aka 3 of 6
print("0.5 of 1: ",env.getValue(xPos=0.5, normalizedX=True))
# get the normalized y-value at x = 0.5 of 6
print("y-norm: ",env.getValue(xPos=0.5, normalizedY = True))
# get the normalized y-value at normalized x
print("y-norm @ x-norm: ",env.getValue(xPos=0.5, normalizedX=True, normalizedY
= True))
```

⁶Keine vertieften Mathekenntnisse nötig! Kurzgesagt, umso höher die Ordnung, desto komplexere Auf- und-Ab sind darstellbar. Generell gilt: Es kann nur sichergestellt werden, dass alle Punkte auf der Kurve liegen, wenn die Ordnung der Kurve der Anzahl der Punkte entspricht.

```
##### OUTPUT
#####
# 0.5 of 6: 0.5
# 0.5 of 1: 3.0
# y-norm: 0.16666666666666666
# y-norm @ x-norm: 1.0
```

Wird ein Wert außerhalb des Wertebereichs abgefragt, wird entweder per Standard interpoliert, alternativ kann der letzte gültige Wert ausgegeben werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1,2,3,2,1,0])
print(env.getValue(11))
print(env.getValue(11, interpolateOutsideOfRange=False))
```

```
##### OUTPUT
#####
# -5.0
# 0.0
```

Werden bei der Konstruktion des Objekts Polynome berechnet, können alternativ auch diese innerhalb und außerhalb des Wertebereiches abgerufen werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1,2,3,1,2,4], polyFit=True)
print("just x: ",env.getValue(0.5, normalizedX=True))
print("1st order: ",env.getValue(0.5, normalizedX=True,polyDegree=1))
print("2nd order: ",env.getValue(0.5, normalizedX=True,polyDegree=2))
print("3rd order: ",env.getValue(0.5, normalizedX=True,polyDegree=3))
print("4th order: ",env.getValue(0.5, normalizedX=True,polyDegree=4))
print("5th order: ",env.getValue(0.5, normalizedX=True,polyDegree=5))
```

```
##### OUTPUT
#####
# just x: 3.0
# 1st order: 1.8571428571428568
# 2nd order: 1.9047619047619047
# 3rd order: 1.904761904761903
# 4th order: 2.3722943722943786
# 5th order: 2.372294372294444
```

4.3 Envelope anzeigen

Mittels `.display()` kann die Envelope unter Zuhilfenahme von `matplotlib` dargestellt werden. Wird `show_polynomials=True` gesetzt, können außerdem die Polynome angezeigt werden.

```
from GreasyPidgin.Envelope import Envelope

env = Envelope([0,1.5,1,3,1,2,4], polyFit=True)
env.display(show_polynomials=True)
```


5. Grid

Ein Grid (de: *Raster*) ist ein set, das zur Quantisierung von Daten genutzt werden kann. Die grundlegenden Methoden um ein Grid zu erzeugen sind Übersetzungen der Array-Konstrukturen aus SuperCollider⁷.

5.1 Ein Grid erzeugen

5.1.1 Direktes Initialisieren

Ein Grid kann im einfachsten Fall direkt aus einem set, tuple oder list erzeugt werden. Dabei ist allerdings zu beachten, dass Grid ein set ist und daher jeder Wert einzigartig sein muss und ein list-Objekt nicht verschachtelt sein darf.

```
from GreasyPidgin.Grid import Grid

print(Grid([1, 3, 5, 7]))
print(Grid({2,4,6,8}))
print(Grid((1,2,3)))

##### OUTPUT
#####
#> Grid(size=4, range=(1, 7), preview=[1, 3, 5, 7])
#> Grid(size=4, range=(2, 8), preview=[2, 4, 6, 8])
#> Grid(size=3, range=(1, 3), preview=[1, 2, 3])
```

5.1.2 Die arithmetische Reihe (1,2,3,4,...)

Für die meisten Anwendungsfälle ist eine einfache arithmetische Reihe, die passende Art und Weise ein Grid zu erzeugen. Via `.series(start, step, size)` wird ein Grid erzeugt, das vom Startwert start ausgehend size - 1 mal den Wert step aufsummiert.

```
from GreasyPidgin.Grid import Grid

print(Grid.series(0, 1, 10))
print(Grid.series(0, 0.1, 5))

##### OUTPUT
#####
#> Grid(size=10, range=(0, 9), preview=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
#> Grid(size=5, range=(0.0, 0.4), preview=[0.0, 0.1, 0.2, 0.3, 0.4])
```

5.1.3 Die geometrische Reihe (1, 2, 4, 8,...)

In einer geometrischen Reihe haben zwei benachbarte Reihenglieder stets das gleiche Verhältnis. Via `.geom(start, ration, size, *, decimals = 100)` wird die Reihe konstruiert, indem vom Startwert start ausgehend der letzte Wert der Reihe iterativ mit der ratio multipliziert wird.

```
from GreasyPidgin.Grid import Grid

print(Grid.geom(1, 2, 8))

##### OUTPUT
#####
#> Grid(size=8, range=(1, 128), preview=[1, 2, 4, 8, 16, 32, 64, 128])
```

Diese erlaubt es zum Beispiel die Frequenzen für *equal distance octave (EDO)*-Stimmungen zu berechnen. Mittels des Methodenslots `decimals` lassen sich zusätzlich die Nachkommastellen durch runden reduzieren.

```
from GreasyPidgin.Grid import Grid

print(Grid.geom(442, 2**((1/12)), 13, 2))
```

⁷Siehe <https://docs.supercollider.online/Classes/Array.html>

```
##### OUTPUT
#####
#> Grid(size=13, range=(442.0, 884.0), preview=[442.0, 468.28, 496.13, 525.63,
556.89, 590.0, 625.08, 662.25, 701.63, 743.35, 787.55, 834.38, 884.0])
```

5.1.4 Initialisieren durch lineare Interpolation

Komplementär zur arithmetischen Reihe, bei der die Schrittgröße aber kein Endwert festgelegt ist, erzeugt `.linearInterpolation(start, end, size, *, decimals = 100, includeEnd = True)` ein Grid, indem die Schrittgröße aus `start`, `end` und `size` bestimmt wird. Mittels des Methodenslots `decimals` lassen sich zusätzlich die Nachkommastellen durch runden reduzieren, mittels `includeEnd` lässt sich wählen, ob der Endwert Teil der Reihe ist oder nicht.

```
from GreasyPidgin.Grid import Grid

print(Grid.linearInterpolation(0, 10, 5))
print(Grid.linearInterpolation(0, 10, 5, includeEnd= False))
print(Grid.linearInterpolation(0, 2.111, 5))
print(Grid.linearInterpolation(0, 2.111, 5, decimals=1))
```

```
##### OUTPUT
#####
#> Grid(size=5, range=(0.0, 10.0), preview=[0.0, 2.5, 5.0, 7.5, 10.0])
#> Grid(size=5, range=(0.0, 8.0), preview=[0.0, 2.0, 4.0, 6.0, 8.0])
#> Grid(size=5, range=(0.0, 2.111), preview=[0.0, 0.52775, 1.0555, 1.58325,
2.111])
#> Grid(size=5, range=(0.0, 2.1), preview=[0.0, 0.5, 1.1, 1.6, 2.1])
```

5.1.5 Fibonacci-artiges Grid

`.fib(size, a, b)` erlaubt es ein Fibonacci-artiges Grid zu erstellen, bei dem der nächste Wert immer die Summe beider vorherigen Werte ist. Initialisiert man mit Wertepaaren aus der Fibonacci-Reihe – z.B.: `a= 1, b = 2` – erhält man die Fortsetzung innerhalb der Fibonacci-Reihe⁸.

```
from GreasyPidgin.Grid import Grid

print(Grid.fib(10,1,2))

##### OUTPUT
#####
#> Grid(size=10, range=(1, 89), preview=[1, 2, 3, 5, 8, 13, 21, 34, 55, 89])
```

`.fib` ist allerdings nicht auf `int` begrenzt sondern erlaubt auch `float`-Werte.

```
from math import e, pi
from GreasyPidgin.Grid import Grid

print(Grid.fib(5,e, pi))

##### OUTPUT
#####
#> Grid(size=5, range=(2.718281828459045, 14.861341617687469),
preview=[2.718281828459045, 3.141592653589793, 5.859874482048838,
9.00146713563863, 14.861341617687469])
```

5.1.6 Initialisieren mit Zufallswerten

Man mag sich fragen, warum man es notwendig sein kann Daten auf Zufallswerte zu quantisieren. Aber Grid erlaubt eine Initialisierung mit Zufallswerten, sowohl `int` als auch `float` basiert. Sollen `int`-Werte erzeugt werden, muss dieses explizit über das Argument `integer = True` ausgedrückt werden.

⁸**Achtung!** Auch hier werden doppelte Werte gelöscht! Die tatsächliche Fibonacci-Reihe [0,1,1,2,3,5,8,...] lässt sich nicht als set darstellen!

```

from GreasyPidgin.Grid import Grid

print(Grid.rand(5,0,12.77))
print(Grid.rand(5,0,12.77,integer=True))
print(Grid.rand(5,0,13))
print(Grid.rand(5,0,13,integer=True))

##### OUTPUT
#####
### als Beispiel, duh!
#> Grid(size=5, range=(0.2856196224877912, 7.789117475083042),
preview=[0.2856196224877912, 4.079976016998381, 5.363352927568584,
6.151736837079151, 7.789117475083042])
#> Grid(size=5, range=(2, 11), preview=[2, 8, 9, 10, 11])
#> Grid(size=5, range=(0.44959703668048456, 7.894089566506886),
preview=[0.44959703668048456, 3.3165137492950127, 3.822080408499572,
7.612798394949432, 7.894089566506886])
#> Grid(size=5, range=(2, 11), preview=[2, 3, 5, 9, 11])

```

5.1.7 Initialisieren über eine Funktion

Grid.fill(size, func_or_value) erlaubt es ein Grid mit einer Konstanten⁹ oder mittels einer Funktion zu füllen. Die Funktion kann dabei als anonyme Funktion definiert werden¹⁰. Dies ist vor allem eine Legacy-Option, die in der Praxis eher selten zur Anwendung kommt, und durch das direkte Initialisieren durch den Output einer Funktion in Form von set, tuple oder list abgelöst wird.

```

from GreasyPidgin.Grid import Grid

print(Grid.fill(10, lambda i: 2**i))

##### OUTPUT
#####
#> Grid(size=10, range=(1, 512), preview=[1, 2, 4, 8, 16, 32, 64, 128, 256,
512])

```

5.2 Quantisieren

Wie eingangs erwähnt dient Grid vor allem dazu Daten zu quantisieren. Mittels der Methode .quantise(value) ist dieses möglich. Es können nur int und float quantisiert werden. Sollen ein iterable wie set, list oder tuple quantisiert werden, muss sichergestellt sein, dass diese nur Zahlen enthalten.

```

from GreasyPidgin.Grid import Grid

g = Grid.series(0,0.5, 10)
print(g.quantise(1.23456789))
print(g.quantise([1.2345, 2.3456]))
print(g.quantise((1.2345, 2.3456)))
print(g.quantise({1.2345, 2.3456}))

##### OUTPUT
#####
#> 1.0
#> [1.0, 2.5]
#> (1.0, 2.5)
#> {1.0, 2.5}

```

5.3 Sieben

Eine weitere Funktion von Grid ist die Möglichkeit ein Sieb auf sich selbst anzuwenden¹¹. Die Siebtheorie selbst stammt aus der analytischen Zahlentheorie und bezeichnet eine

⁹**Achtung!** Es wird ein grid mit der Länge 1 erzeugt!

¹⁰Siehe https://www.w3schools.com/python/python_lambda.asp

¹¹Für einen Deepdive des Ursprungs dieser Idee siehe: <https://www.iannis-xenakis.org/en/sieve-theory/>

Reihe von Techniken, mittels derer eine Menge gefiltert wird, so dass am Ende nur die Zahlen enthalten sind, die nicht durch das Sieb gefallen sind.

5.3.1 Hintergrund: Das Sieb des Eratosthenes

Das älteste dokumentierte Sieb, das auch als solches bezeichnet wurde ist das Sieb des Eratosthenes aus dem 3. Jhd vor Christus. Dieses Sieb dient dazu Primzahlen aus einer beliebigen Menge natürlicher Zahlen herauszufiltern. Im ersten Schritt werden dafür alle Zahlen geordnet auf einem Zahlenstrang notiert. In Python entspricht das einer Liste, die mittels `[i+2 for i in range(100-2)]` erzeugt wird. Ausgehend von der 2 als kleinste Primzahl werden via Modulo alle ganzzahligen Vielfachen von 2 aus der Liste entfernt und die zwei als letzte Primzahl an die Liste `primes` gehängt. Die nächste größere Zahl 3 *muss* nun wieder auch eine Primzahl sein.

```
allIntegers = [i+2 for i in range(100-2)]
lenTS = len(allIntegers)
primes = []
```

```
while lenTS > 0:
    newPrime = allIntegers[0]
    primes.append(newPrime)
    for i in allIntegers:
        if i%newPrime == 0:
            allIntegers.remove(i)
    lenTS = len(allIntegers)

print(primes)
```

OUTPUT

#####

```
#> [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67,
71, 73, 79, 83, 89, 97]
```

Diese einfachste Form des Siebs basiert nur auf einem Parameter, der Schrittgröße, in Form der aufsteigenden Primzahlen.

5.3.2 Xenakis

Iannis Xenakis entwickelte seine Form der Siebtheorie während seiner Zeit in Westdeutschland um 1963. Im Gegensatz zu Eratosthenes, ging es Xenakis nicht darum, Primzahlen zu bestimmen, sondern vor allem eine sortierte Menge (*well sorted set*) so zu filtern und mit anderen Filtrierungen der gleichen Menge so zu kombinieren, dass eine neuartige mathematisch fundierte harmonische Sprache entsteht, die sich an keiner bisher etablierten Harmonik – weder tonaler, noch serieller – orientiert. Neu in Xenakis' Denken ist dabei die Einführung eines Offsets, ab dem die Menge gefiltert wird.

```
from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =0)
g.sieve(stepSize = 3, offsetIndex =1)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])
```

OUTPUT

#####

```
#> Grid(size=7, range=(0, 18), preview=[0, 3, 6, 9, 12, 15, 18])
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
```

5.3.3 Erweiteretes Sieben

In der Implementation von `Grid.sieve()` gibt es nun die Möglichkeit die traditionellen Siebverfahren als auch Erweiterungen dieser zu nutzen. Mittels `stepSize` und `offsetIndex` lassen sich alle Siebe nach Xenakis anwenden. Wird ein Grid gesiebt, wird das Ergebnis im Dictionary des Slots `Grid.sieved` gespeichert. Innerhalb von `Grid` zählt eine Variable hoch, wie oft bereits gesiebt wurde; dieses wird als key für das dict genutzt.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, discardOffset= False)
for key in g.sieved:
    print(key)

##### OUTPUT
#####
#> sieved_0
#> sieved_1

```

Zusätzlich erlaubt discardOffset die sonst übersprungen Indices in das Resultat mit aufzunehmen.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, discardOffset= False)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
#> Grid(size=8, range=(0, 19), preview=[0, 1, 4, 7, 10, 13, 16, 19])

```

Mittels inverse wird das Sieb umgekehrt. In dieser komplementären Filtrierung werden also alle Werte ausgegeben, die nicht im Sieb hängenbleiben.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =1)
g.sieve(stepSize = 3, offsetIndex =1, inverse=True)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=7, range=(1, 19), preview=[1, 4, 7, 10, 13, 16, 19])
#> Grid(size=12, range=(2, 18), preview=[2, 3, 5, 6, 8, 9, 11, 12, 14, 15, 17, 18])

```

reverse wiederum erlaubt die Filtrierung in umgekehrter Reihenfolge durchzuführen. Da das Umkehren des Inputs vor der Offset-Operation erfolgt, lassen sich damit Filtrierungen erzeugen, die nicht auf das Ende eines Grid angewendet werden.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset = False)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset = False, reverse=True)
for s in g.sieved:
    print(g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> Grid(size=10, range=(0, 17), preview=[0, 1, 2, 3, 4, 5, 8, 11, 14, 17])
#> Grid(size=10, range=(2, 19), preview=[2, 5, 8, 11, 14, 15, 16, 17, 18, 19])

```

Mittels des Parameters rotation lässt sich eine weitere Offset-Operation durchführen, nachdem offsetIndex und reverse angewendet wurden.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False, rotation=1)

```

```

g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False, reverse=True)
g.sieve(stepSize = 3, offsetIndex =5, discardOffset=False,
reverse=True,rotation=1)

for s in g.sieved:
    print(s," : ",g.sieved[s]['sievedGrid'])

##### OUTPUT
#####
#> sieved_0 : Grid(size=10, range=(0, 17), preview=[0, 1, 2, 3, 4, 5, 8, 11,
14, 17])
#> sieved_1 : Grid(size=10, range=(0, 18), preview=[0, 1, 2, 3, 4, 6, 9, 12,
15, 18])
#> sieved_2 : Grid(size=10, range=(2, 19), preview=[2, 5, 8, 11, 14, 15, 16,
17, 18, 19])
#> sieved_3 : Grid(size=10, range=(1, 19), preview=[1, 4, 7, 10, 13, 15, 16,
17, 18, 19])

```

Das dict enthält außerdem alle Informationen, über das angewendete Sieb.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
g.sieve(stepSize = 3, offsetIndex =5)
key = 'sieved_0'
sieved = g.sieved[key]
print(key)
for param in sieved:
    print(f"{param}: {sieved[param]}")

##### OUTPUT
#####
#> sieved_0
#> sievedGrid: Grid(size=5, range=(5, 17), preview=[5, 8, 11, 14, 17])
#> stepSize: 3
#> offsetIndex: 5
#> discardOffset: True
#> inverse: False
#> reverse: False
#> rotation: 0

```

5.4 Sampling

Neben dem Sieben ist Sampling eine weitere Möglichkeit, Elemente aus einem Grid auszuwählen. Sampling ruft die `random.sample()`-Funktion auf und wählt zufällig `numElements` Elemente aus dem Grid aus und erzeugt ein neues Grid mit dieser Auswahl.

```

from GreasyPidgin.Grid import Grid
g = Grid.series(0, 1, 20)
for _ in range(5):
    g.sample(5)
for i in range(5):
    print(g.sampled['sampled_'+str(i)])

```


6. TimeGrid

Ein TimeGrid dient dazu Daten in der Zeit zu quantisieren oder eine Menge von Zeitdaten zu generieren. Dies ist zuallererst mit den Grid-Konstrukturen möglich.

6.1 Vererbte Grid-Konstrukturen

Die implementierten vererbten Konstrukturen sind die direkte Erzeugung via *iterable*, *.series()* oder *.interpolate()*. Die anderen Konstrukturen sind zur Zeit noch nicht vollständig implementiert und sollten nur mit Vorsicht verwendet werden.

6.1.1 Direktes Initialisieren

Ein TimeGrid kann direkt mit Sekundenwerten initialisiert werden.

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid([0,1,2,3,4.5], bpm = 111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [0, 1, 2, 3, 4.5],
#> durationSec = 4.5,
#> durationBeats = 8.325,
#> BPM         = 111,
#> size        = 5,
#> layers:
```

6.1.2 Lineare Konstruktion

Via *.series(startTimeSec, stepSizeSec, size)*

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid.series(1, 1, 10, bpm = 111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
#> durationSec = 10.0,
#> durationBeats = 18.5,
#> BPM         = 111,
#> size        = 10,
#> layers:
```

6.1.3 Lineare Interpolation

Via *.linearInterpolation(startTimeSec, endTimeSec, size)*

```
from GreasyPidgin.TimeGrid import TimeGrid

t = TimeGrid.linearInterpolation(startTimeSec=0, endTimeSec= 2, size= 11, bpm
=111)
print(t)

##### OUTPUT
#####
#> TimeGrid
#> sorted      = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8, 2.0],
#> durationSec = 2.0,
#> durationBeats = 3.7,
#> BPM         = 111,
#> ticksPerBeat = 480,
```

```
#> size = 11,
#> layers:
```

6.1.4 Nicht vollständig

```
from GreasyPidgin.TimeGrid import TimeGrid
```

```
tr = TimeGrid.rand(10, 0, 10)
tg = TimeGrid.geom(1, 1.1, 10)
tf = TimeGrid.fib(10, 1, 2)
tl = TimeGrid.fill(10, lambda i: 2**i)
```

6.2 Mikrotime-basierte Konstruktoren

Die zwei vorrangigen Arten ein TimeGrid zu erzeugen sind allerdings die Beat-bezogenen Methoden `.fromSubdivision()` und `.concatenateRQQEvents()`. Beide Methoden folgen der Idee, eine Makrozeit in eine Mikrozeit zu teilen.

6.2.1 Multi-Mikrotime Teilung

Mit der Methode `fromSubdivisions(subdivisions, durationBeats)` kann eine vorab bestimmte Anzahl von Beats gleichzeitig in mehrere Untereinheiten unterteilt werden. Für jede Unterteilung wird der Resultierende Wert in ein separates Layer geschrieben, als auch dem TimeGrid selbst beigefügt. Dauern und Endpunkte, sowohl in Sekunden als auch Beats, werden automatisch berechnet.

```
from GreasyPidgin.TimeGrid import TimeGrid
```

```
t = TimeGrid.fromSubdivisions(subdivisions=[3,4], durationBeats=10)
print(t)
```

```
##### OUTPUT
#####
#> TimeGrid
#> sorted = [0.0, 0.25, 0.333333, 0.5, 0.666667, 0.75, 1.0, 1.25,
1.333333, 1.5, 1.666667, 1.75],
#> durationSec = 1.75,
#> durationBeats = 1.75,
#> BPM = 60.0,
#> ticksPerBeat = 480,
#> size = 12,
#> layers:
#> / 3 : [0.0, 0.333333, 0.666667, 1.0] ...
#> / 4 : [0.0, 0.25, 1.0, 0.5] ...
```

6.2.2 RQQ Teilung

RQQ ist eine Rhythmus-Notation, die von William “Bill” Gardner Schottstaedt als Teil von *Patchwork* / *OpenMusic* entwickelt wurde.¹² Die Idee hinter RQQ ist, dass eine beliebige verschachtelte Rhythmusstruktur mittels simpler Proportionen abgebildet werden kann. In dieser Python-Implementation kann ein RQQ-Objekt entweder alleinstehend verwendet werden oder als Konstruktor für ein TimeGrid.

Nehmen wir an, eine 4telnote entspricht einem Beat, so können wir eine 8eltriole wie folgt darstellen:

```
from GreasyPidgin.TimeGrid import RQQ
```

```
print(RQQ(1, [1,1,1]))
```

```
##### OUTPUT
#####
#> RQQ
```

¹²Originalquellen: <http://recherche.ircam.fr/equipes/repmus/PatchWork/>, <https://github.com/openmusic-project>; Implementation von Michael Edwards in `slippery` klicken: <https://michael-edwards.org/sc/manual/rhythms.html#tuplets>

```
#> durationBeats      : 1
#> subdivisions       : [1, 1, 1]
#> bpm                : 81.0
#> startOffsetBeats   : 0.0
#> startsBeats        : [0.0, 0.3333333333333333, 0.6666666666666666]
```

Mittel des Rest-Objekts¹³ können Pausen hinzugefügt werden. Eine 4telnote gefolgt von einer 8elpause und ein 8elnote sieht wie folgt aus:

```
from GreasyPidgin.TimeGrid import RQQ, Rest
```

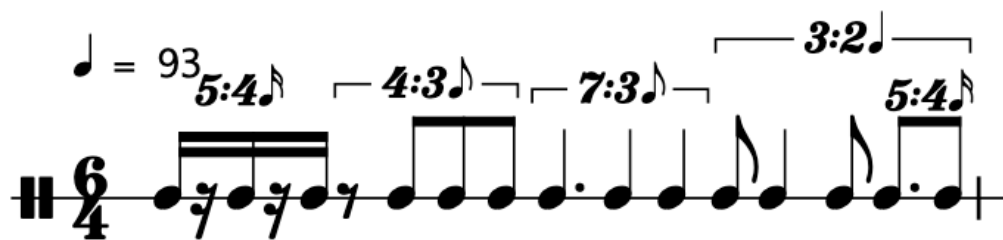
```
print(RQQ(2, [2,Rest(1),1]))
print("Rest == R: ",Rest(1) == R(1))
##### OUTPUT
#####
#> RQQ
#> durationBeats      : 2
#> subdivisions       : [2, 1.0, 1]
#> bpm                : 60.0
#> startOffsetBeats   : 0.0
#> startsBeats        : [0.0, 1.5]
#
#> Rest == R:  True
```

So können nun beliebig komplizierte Verästelungen geschrieben werden.

```
from GreasyPidgin.TimeGrid import RQQ, R, TimeGrid
```

```
rqq = RQQ(6,[
    RQQ(1, [1,R(1),1,R(1),1]),
    RQQ(1.5, [R(1),1,1,1]),
    RQQ(1.5, [3,2,2]),
    RQQ(2, [
        RQQ(2,[1,2,1]),
        RQQ(1, [3,2])
    ])
])

t = TimeGrid.fromRQQ(rqq, bpm = 93)
print(t)
t.parseMidi()
```



6.3 Midi-Export

Wie im vorigen Beispiel bereits durchgeführt, kann mit der Methode `.parseMidi()` das Grid, sowie alle Layer als Multi-Track Midifile exportiert werden.

¹³Als Kurzschreibweise ist auch R möglich

7. PitchGrid

Ein weiteres spezialisiertes Grid ist das PitchGrid. Es dient dazu Rohdaten in ein tonales System zu überführen oder generell ein harmonisches System zu erzeugen. Neben den vererbten Konstruktoren gibt es vor allem die Möglichkeit auf bestehende Intonations und Skalensysteme via `.fromScaleMaskObj()` zuzugreifen. Alternativ erlaubt der Konstruktor `.fromCorrelation()` aus einer Reihe von Möglichen Kandidaten das Stimmungssystem auszuwählen, das am besten auf die angegebenen Tönhöhen passt.

7.1 Stimmungssysteme

In der Datei `TuningSystemAndScaleMask.py` findet sich eine Sammlung von Skalen und Masken, mit denen sich eine Vielzahl verschiedener hamonischer Systeme abbilden lässt.

7.1.1 Oktavierende Stimmungssysteme

Die meisten musikalisch verwendeten Stimmungs- und Skalensystem bauen darauf auf, dass sie oktavtransponierend sind, das heißt, dass jede Tonhöhe potenziell eine beliebige Anzahl von Oktaven rauf und runter transponiert werden kann ohne den Charakter der Skala zu verändern.

7.1.1.1 Gleichschwebend

Als gleichschwebend bezeichnet man Stimmungssysteme in denen die Verhältnisse zwischen benachbarten Frequenzen immer die selben sind. Das in der westlichen Musik am weitesten verbreitete und standardisierte System ist die gleichschwebende Unterteilung der Oktave in 12 Schritte. Auf english: *12 Step Equal Distance Octave* oder kurz *edo12*. Digital ist allerdings die Unterteilung der Oktave in eine beliebige Anzahl von Schritten möglich. GreasyPidgin bietet standardmäßig nicht nur *edo12*, sondern zusätzlich *edo7*, *edo11*, *edo53* sowie *edo72*.

7.1.1.2 Nicht gleichschwebend - aber 12-tönig

Historisch gesehen stellen gleichschwebende System allerdings eine Minderheit dar. Als Alternative zu *edo12* beinhaltet GreasyPidgin daher die Möglichkeit Skalen in der folgenden Stimmungssystemen zu erzeugen: *werckmeister1*¹⁴, *werckmeister2*¹⁵, *werckmeister3*¹⁶, *werckmeister4*¹⁷, *kirnberger2*¹⁸, *kirnberger3*¹⁹, *meantoneQuarterComma*²⁰, *pythagorean*²¹.

Aus dem Bereich der *Just Intonation*-Systeme ist bislang ausschließlich *Zeta12*²² implementiert.

7.1.1.3 Weder gleichschwebend, noch 12-tönig

Neben diesen historischen europäischen Intonationssystemen ist mit *shruti22* auch die Grundlage für Skalen nach dem Vorbild von Ragas aus der karnatischen Tradition implementiert.²³

In Zukunft sollen auch noch weitere Systeme implementiert werden, wie zum Beispiel Pélog, Slendro, PartchJustIntonation und andere.²⁴

¹⁴Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T016.html

¹⁵Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T017.html

¹⁶Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T018.html

¹⁷Bezeichnet nach: http://www.groenewald-berlin.de/text/text_T019.html

¹⁸nach <https://www.lehrklaenge.de/PHP/Tonsystem/KirnbergerStimmung.php>

¹⁹ebenda

²⁰https://de.wikipedia.org/wiki/Mitteltönige_Stimmung

²¹Pythagoreische Stimmung

²²Ratios von <https://en.xen.wiki/w/Zeta12>

²³siehe: <https://www.carnaticcorner.com/articles/sruthis.html> alternativ ist eine Erweiterung möglich: https://en.xen.wiki/w/A_shruti_list

²⁴Bei Interesse zur eigenen Implementation und Mitwirkung bietet das XenharmonicWiki einen guten Recherche-Einstieg: https://en.xen.wiki/w/Gallery_of_12-tone_just_intonation_scales

7.1.2 Nicht-oktavierende Stimmungssysteme

z.B.: Bohlen-Pierce ; bisher nicht implementiert

7.2 Skalen

GreasyPidgin erlaubt dabei eine hohe theoretische Flexibilität. Die bisher implementierten Skalen sind:

7.2.1 12-Ton-Basis

7.2.1.1 Kirchentonarten

- ionian, dorian, phrygian, lydian, mixolydian, aeolian, locrian

7.2.1.2 Kirchentonarten in harmonisch Moll

- harmonicMinor, locrianSharp6, ionianSharp5, dorianSharp4, phrygianDominant, lydianSharp2, ultraLocrian

7.2.1.3 Kirchentonarten in melodisch Moll

- melodicMinor, dorianFlat2, lydianAugmented, lydianDominant, mixolydianFlat6, locrianSharp2, altered

7.2.1.4 Modi mit begrenzten Transpositionsmöglichkeiten

- chromatic, wholeTone, halfToneWholeTone, messiaen3, messiaen4, messiaen5, messiaen6, messiaen7

7.2.1.5 Pentatoniken

- majorPentatonic, minorPentatonic, suspendedPentatonic, bluesMinorPentatonic, bluesMajorPentatonic

7.2.2 22-Shruti-Basis

7.2.2.1 Skalen nach hindustanischem Vorbild²⁵

- abhogi, mohanam, hamsadhwani, hindolam, shuddhaSaveri, madhyamavati, khamas, revati, kalyani, todi, bhairavi, shankarabharanam

7.2.2.2 Skalen nach karnatischem Vorbild

- n.n.²⁶

7.2.3 EDO53-Basis

7.2.3.1 Türkische Maqams

- n.n.

7.2.4 EDO72-Basis²⁷

7.2.4.1 Iranische Dastgāhs und Āwāz

- n.n.

²⁵Wichtige Anmerkung für alle Weißbrote: Es sind ausdrücklich Skalen nach dem *VORBILD* von Ragas, da der Begriff Raga mehr als nur eine Abfolge von Tonhöhen bezeichnet. Möchte man in diesem Klangraum arbeiten, empfehle ich Kontakt zu Musiker!nnen aufzunehmen, die sich in diesen Traditionen bewegen und mit denen man gemeinsam ein Verständnis für diese Klangsprache entwickelt. Das gleiche gilt auch für die weiteren Systeme von *Maqams* und *Dastgāhs*.

²⁶To be implemented: [Melakarta#Table of Melakarta ragas](#)

²⁷Zur Zeit ist noch nicht klar, ob und in welchem EDO-System ich die arabischen Maqams implementieren will. Unter https://en.xen.wiki/w/Middle-Eastern_music finden sich neben edo72 verschiedene Möglichkeiten.

7.3 Konstruktoren

7.3.1 Skalenerzeugung

`.fromScaleMaskObj()` bietet die Möglichkeit via Schlagwort eine Skala zu erzeugen. Der Parameter `mask` erwartet einen Skalennamen²⁸, `tuningSystem` erwartet ein zum Skalennamen passendes Stimmungssystem. Mit den Parametern `lowestPitchMidi` und `highestPitchMidi` lässt sich der maximale Ambitus des `PitchGrid` bestimmen.

```
from GreasyPidgin.PitchGrid import PitchGrid

p = PitchGrid.fromScaleMaskObj(
    mask= 'chromatic',
    tuningSystem= "edo12",
    lowestPitchMidi=45,
    highestPitchMidi=55)
print(p)

##### Output
#####
#> PitchGrid
#>   size      : 11,
#>   range      : 45.0 to 55.0,
#>   tuning     : 'edo12',
#>   scale      : 'chromatic',
#>   pitchClass : [0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0,
11.0],
#>   preview    : [45.0, 46.0, 47.0, 48.0, 49.0, 50.0, 51.0, 52.0, 53.0,
54.0 ...
```

Für alle Stimmungssysteme, die nicht `edo12` oder Vielfache dessen (`edo24`, `edo72`, etc.) sind, ist es notwendig, den Startpunkt `tuningSystemRoot` für die Berechnung des Systems festzulegen.

```
from GreasyPidgin.PitchGrid import PitchGrid

p60 = PitchGrid.fromScaleMaskObj(tuningSystem= "edo7",tuningSystemRoot = 60)
p59 = PitchGrid.fromScaleMaskObj(tuningSystem= "edo7",tuningSystemRoot = 59)
print(p60)
print(p59)

# same PitchClass (edo7) but different resulting pitches due to different
# calculation start. The default state for all systems is "chromatic" but
# only
# edo12 produces a real chromatic scale. For all other tuning systems
# "chromatic" tries to find the closest pitch in the system that resembles a
# edo12 chromatic pitch. This doesn't work for shruti based systems!

##### Output
#####
# PitchGrid
#   size      : 8,
#   range      : 60.0 to 72.0,
#   tuning     : 'edo7',
#   scale      : 'chromatic',
#   pitchClass : [0.0, 1.71, 3.43, 5.14, 6.86, 8.57, 10.29],
#   preview    : [60.0, 61.71, 63.43, 65.14, 66.86, 68.57, 70.29, 72.0

# PitchGrid
#   size      : 7,
#   range      : 60.71 to 71.0,
#   tuning     : 'edo7',
```

²⁸Eine *mask* ist hier am besten mit den deutschen Wort *Schablone* zu übersetzen – ein Filter mittels welchem aus einem vollständigen chromatischen set nur die Töne ausgewählt werden, die für die Skala benötigt werden.

```
# scale      : 'chromatic',
# pitchClass : [0.0, 1.71, 3.43, 5.14, 6.86, 8.57, 10.29],
# preview    : [60.71, 62.43, 64.14, 65.86, 67.57, 69.29, 71.0]
```

Noch deutlicher fällt dies bei historischen westlichen Stimmungen ins Gewicht. So lässt sich eine dorische Tonleiter in mitteltöniger Stimmung mit dem Grundton C sowohl in einem System konstruieren, das von C ausgehend konstruiert ist, oder zum Beispiel von Cis aus.

```
from GreasyPidgin.PitchGrid import PitchGrid
```

```
pg60 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
    tuningSystemRootMidi=60,
    scaleRootMidi=60,
    highestPitchMidi=73
)
```

```
pg61 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
    tuningSystemRootMidi=61,
    scaleRootMidi=60,
    highestPitchMidi=73
)
```

```
print(pg60)
print(pg61)
```

```
##### Output
```

```
#####
```

```
#> PitchGrid
#> size      : 8,
#> range     : 60.0 to 72.0,
#> tuning    : 'meantoneQC',
#> scale     : 'dorian',
#> pitchClass : [0.0, 1.93, 3.1, 5.03, 6.97, 8.9, 10.07],
#> preview   : [60.0, 61.93, 63.1, 65.03, 66.97, 68.9, 70.07, 72.0]
```

```
# PitchGrid
#> size      : 8,
#> range     : 60.12 to 72.12,
#> tuning    : 'meantoneQC',
#> scale     : 'dorian',
#> pitchClass : [0.12, 1.76, 2.93, 4.86, 6.79, 9.14, 9.9],
#> preview   : [60.12, 61.76, 62.93, 64.86, 66.79, 69.14, 69.9, 72.12]
```

7.4 Export

Um ein PitchGrid darzustellen, lässt es sich wie ein TimeGrid via `.parseMidi()` als MIDI-Datei exportieren²⁹. Für die Verwendung in DAWs werden die mikrotonalen Abweichungen in PitchBend-Werte umgerechnet, für eine Notation werden sie als lyrics an den Noten annotiert.

```
from GreasyPidgin.PitchGrid import PitchGrid
```

```
pg60 = PitchGrid.fromScaleInTUNING_SYSTEMS(
    'dorian',
    tuningSystemKey='meantoneQC',
    tuningSystemRootMidi=60,
    scaleRootMidi=60,
    highestPitchMidi=73
)
```

²⁹Beim Export als Midi muss allerdings die PitchBendRange auf das MIDI-Instrument abgestimmt sein, auf dem es wiedergegeben wird! Siehe `TemporalObject.parseMidi()` um ggf. eine Kalibrierungsdatei zu erzeugen.

```
pg60.parseMidi("/tmp/mqc_dorian_tsRoot60")
```

```
##### Output
```

```
#####
```

```
#> Midi file saved to: /tmp/
```

```
mqc_dorian_tsRoot60PitchGrid_meantoneQC_dorian.mid
```



8. DynamicGrid

Ein DynamicGrid erlaubt das Rastern von Dynamikwerten und die Übersetzung von reellen Lautstärken von Instrumenten in Dezibel in Midi-Werte.

```
from GreasyPidgin.DynamicGrid import DynamicGrid

d1 = DynamicGrid()
for entry in d1.dynamicMapping:
    print(entry, d1.dynamicMapping[entry])
```


9. TemporalObject

9.1 Was ist ein TemporalObject?

Ein TemporalObject – kurz T0 – ist die Superklasse für alle Objekte, die in der Zeit organisiert werden können / müssen. Das T0 vererbt alle Eigenschaften an

Phonon: Ein *Klangpartikel* - die kleinste reduzierbare Einheit von Klang.

Gesture: Eine *Klanggeste* - eine Kombination von mehrerer Phonon - Objekte , die eine Sinneinheit bilden.

GreasyPidgin: *Toplevel-Struktur*, die eine algorithmische Generation und Analyse komplexer Strukturen aus Phonon , Gesture, Phrase und Section ermöglicht.

Weitere T0 - Objekt befinden sich gerade noch in der Entwicklungsphase. Zur Zeit geplant sind:

Phrase: Eine abgeschlossene *musikalische Mikrostruktur*, bestehend aus einem oder mehreren Objekten der Typen Phonon und Gesture

Section: Eine abgeschlossene *musikalische Makrostruktur*, ein Abschnitt aus Phonon - , Gesture - und/oder Phrase - Objekten.

Photon : Lichtsteuerdaten (noch nicht implementiert)

Graphon: Videosteuerdaten (noch nicht implementiert)

9.2 Ein TemporalObject erzeugen

Ein T0 muss immer mit einer Startzeit und einer Dauer initialisiert werden.

```
T0 = TemporalObject(startTimeSec=0.0, durationSec=0.0)
```

Als Superklasse kann eine T0-Instanz, weitere T0-Instanzen, sogenannte children beinhalten. Wird ein T0 mit children initialisiert, so werden Dauer und Endzeitpunkt automatisch anhand der Zeitwerte der children bestimmt. Optional kann ein T0 zusätzlich noch mit einem Tempo in beatsPerMinute (bpm) , Hüllkurven (envelopes) und Anmerkungen (annotations) erzeugt werden.

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Envelope import Envelope
```

```
child1 = TemporalObject(
    startTimeSec=0.0, durationSec=1.0,
    envelopes={'vibrato': Envelope([0,1,0,1,0,1,0])})
```

```
child2 = TemporalObject(
    startTimeSec=1.0, durationSec=3.1415,
    annotations=['Hello World!'])
```

```
parentalT0 = TemporalObject(
    startTimeSec=0.0,
    durationSec=0.0,
    bpm=60.0,
    children=[child1, child2])
```

```
print(parentalT0)
```

```
##### OUTPUT
```

```
#####
```

```
#> TemporalObject(id=2, start=0.000, dur=4.142, end=4.142, children=2)
```

Beim Midiexport werden die Anmerkungen automatisch als 'Songtext' exportiert. Ein T0 verfügt über weitere Slots, die den Konventionen der MIDI-Nomenklatur folgen, da der Hauptexportweg der Export als MidiFile ist.

9.3 Ein TemporalObject exportieren

Ein T0 lässt sich mit der Methode `.parseMidi()` schnell als Midifile exportieren. Sind weder Pitch noch Lautstärke gesetzt, fällt die Methode auf die default-Werte zurück:

- `pitchMidi = 127`
- `dynamicdB = 100` aus welchem `velocityMidi = 127` abgeleitet wird.

```
from GreasyPidgin.TemporalObject import TemporalObject
from GreasyPidgin.Envelope import Envelope

T0 = TemporalObject(
    # startTimeSec, durationSec
    0.0, 10.0,
    # all other slots can only be accessed with keywords
    bpm = 71,
    envelopes={
        "modulationWheel": Envelope([0,100,0,90,0,60,127])},
    annotations='wheeee')

T0.parseMidi(
    #standard values
    path = "/tmp/out.mid",
    trackname = "dummy",
    parseEnvelopes= True,
    perNoteTracks= False)

##### OUTPUT
#####
#> Midi file saved to: /tmp/out.mid
```

In der Datei `TemporalObject.py` lassen sich am Ende der Datei die Namen aller Envelope-Slots finden. Der `pitchwheel`-Slot ist allerdings in dieser Version ausschließlich über einen mikrotonalen Midi-Wert zu ändern.

Danger

Es ist darauf zu achten, dass die `pitchBendRange` auf den Synthesizer abgestimmt ist, der das Midifile abspielt! Die meisten Synthesies haben entweder eine $2^{*}10$ oder $2^{*}12$ Bending-Auflösung!

Beispiele aus *Ableton Live 12.3*:

Synth-Plugin	Pitch Bend Range
Analog	$2^{*}12$
Drift	$2^{*}12$
Electric	$2^{*}10$
Meld	$2^{*}10$
Operator	$2^{*}10$
Tension	$2^{*}12$
Wavetable	$2^{*}12$

Um herauszufinden, welche Range der Synth nutzt können mit dem folgenden Code Kalibrierungsdateien erzeugt werden. Der richtige Wert ist gefunden, wenn der Halbton zwischen C und Cis in 10 gleichgroßen Schritten erreicht wird.

```
from GreasyPidgin.TemporalObject import TemporalObject

t = TemporalObject()
children = [
    TemporalObject(i/2, 0.5) for i in range(10)]
```

```

for i,c in enumerate(children):
    c.pitchMidi = 60+(i/10)
    t.add(c)
for pbDivision in [2**10, 2**12]:
    t.parseMidi(
        "/tmp/pitchBendConfig"+str(pbDivision)+".mid",
        str(pbDivision),
        pitchBendRange=pbDivision)

```

```

from GreasyPidgin.TemporalObject import TemporalObject

```

```

T0 = TemporalObject(0.0, 10.0)
T0.pitchMidi = 60.5 #quartertone high c4
T0.parsePitch()
print(T0.annotations)

```

```

##### OUTPUT
#####
#> [(0.0, '50 CENT')]

```

Besteht ein T0 aus beliebig verschachtelte children werden diese automatisch auf eine hierarchische Ebene gebracht. Bei gleichzeitigen Ereignissen mit verschiedenen mikrotonalen Abweichungen ist die Option perNoteTracks auf True zu setzen. In diesem Fall werden dann automatisiert neue MidiTracks erzeugt, sollten sich pitchbend-Befehle widersprechen.

```

from GreasyPidgin.TemporalObject import TemporalObject

```

```

T1 = TemporalObject(0.0, 10.0)
T2 = TemporalObject(1.0, 10.0)
T3 = TemporalObject(0, children = [T1, T2])

#set some pitches
T1.pitchMidi = 60 #natural c
T2.pitchMidi = 70-0.31 # natural septh lowered b flat

T3.parseMidi("/tmp/lastPitchBendOverwritesEverything.mid")
T3.parseMidi("/tmp/newTracksAreGenerated.mid",perNoteTracks=True)

```

```

##### OUTPUT
#####
#> Midi file saved to: /tmp/lastPitchBendOverwritesEverything.mid
#> Midi file saved to: /tmp/newTracksAreGenerated.mid

```


10. Phonon

10.1 Was ist ein Phonon?

Ein Phonon ist ein *Klangpartikel* - die kleinste beschreib- und reduzierbare Einheit von Klang. Der Komplexität sind dabei allerdings keine Grenzen gesetzt.

10.2 Ein Phonon erzeugen

10.2.1 Sekunden oder Beats

Ein Phonon muss wie ein TemporalObject immer mit einer Startzeit und einer Dauer initialisiert werden. Dabei ist es in einem jetzt musikalischen Kontext möglich die Zeit entweder in 'seconds' / 's' (default) oder 'beats' / 'b' zu setzen.

```
from GreasyPidgin.Phonon import Phonon

ps = Phonon(1,duration = 10,timeUnit='s')
pb = Phonon(1,duration = 10,timeUnit='b', bpm = 161)

print(ps)
print(pb)

##### OUTPUT
#####
# Phonon(
#   id      : 1
#   startSec : 1.000, durSec: 10.000, endSec: 11.000
#   pitchesMidi :
#     - all      : {0}
#     - highest   : [0]
#     - lowest    : [0]
#     - mostCommon : [0.0]
#     - leastCommon : [0.0]
#
# [0]
# Phonon(
#   id      : 2
#   startSec : 0.373, durSec: 3.727, endSec: 4.099
#   pitchesMidi :
#     - all      : {0}
#     - highest   : [0]
#     - lowest    : [0]
#     - mostCommon : [0.0]
#     - leastCommon : [0.0]
```

10.2.2 Tonhöhe

Die essentielle Erweiterung ist nun, dass Tonhöhen ergänzt werden können. Die Tonhöhen selbst können dabei entweder als 'midi', 'hz' oder 'spn'³⁰ formatiert sein.

```
from GreasyPidgin.Phonon import Phonon

pSingleMidi = Phonon(1,1,pitch=68)
pMidiMicro = Phonon(1,1,pitch=68.45)

pFromSPN = Phonon(1,4,'af4', pitchUnit = 'spn')

pFromFreq = Phonon(1,2, 440, pitchUnit='hz')

if type(pSingleMidi) == Phonon and type(pMidiMicro) == Phonon:
    print("This is 'True' because pitchUnit='midi' is the default")

print("The midipitch for pFromSPN is: ",pFromSPN.allPitches[0])
```

³⁰scientific pitch notation, siehe Pitch.py, 02_Pitch.md oder **Kapitel 2: Pitch**

```
print("The midipitch for pFromFreq is: ",pFromFreq.allPitches[0])
```

```
##### OUTPUT
#####
# This is 'True' because pitchUnit='midi' is the default
# The midipitch for pFromSPN is: 68.0
# The midipitch for pFromFreq is: 69.0
```

10.2.3 Tonhöhenstruktur

Ein Phonon kann drei verschiedenen Tonhöhenstrukturen nutzen:

- 'seq' / 's': Eine monophone Sequenz von Tonhöhen, inkl. des Spezialfalls Sequenzlänge 1 für nur eine Tonhöhe => *default*. In dieser Version muss die Abfolge von Tonhöhen in gleichen Abständen erfolgen.

```
from GreasyPidgin.Phonon import Phonon
```

```
pSingle = Phonon(1,1,pitch='af4',pitchUnit='spn',)
pList = Phonon(1,1,pitch=['af4'],pitchUnit='spn',)
print(pSingle.allPitches == pList.allPitches, ", because both Phonons are a
one-element 'seq' by default")
```

```
pSeq = Phonon(1,1,pitch=['af4', 'b4', 'bf4', 'b4'],pitchUnit='spn',)
print(pSeq)
```

```
##### OUTPUT
#####
# True , because both Phonons are a one-element 'seq' by default
# Phonon(
#   id          : 2
#   startSec     : 1.000, durSec: 1.000, endSec: 2.000
#   pitchesMidi  :
#     - all      : [68.0, 70.0, 71.0]
#     - highest   : [71.0]
#     - lowest    : [68.0]
#     - mostCommon : [71.0]
#     - leastCommon : [70.0]
```

- 'chord' / 'c': Ein einzelner Akkord bzw ein einzelnes komplexes Spektrum³¹.

```
from GreasyPidgin.Phonon import Phonon
```

```
pChladniSpectrum = Phonon(
    2,10,
    [320, 630, 733, 854, 1370, 1650, 1850, 3159],
    pitchUnit='hz',
    chordOrSeq='chord')
```

```
print(pChladniSpectrum)
```

```
##### OUTPUT
#####
# Phonon(
#   id          : 0
#   startSec     : 2.000, durSec: 10.000, endSec: 12.000
#   pitchesMidi  :
#     - all      : [63.486820576352436, 75.21417965835143,
77.83571609530983, 80.48079055314997, 88.66320557187876, 91.88268714730222,
93.86339810254819, 103.12671009866656]
#     - highest   : [103.12671009866656]
#     - lowest    : [63.486820576352436]
```

³¹Bespiel-Spektrum entnommen aus: https://www.researchgate.net/publication/380052462_Chladni_Plate_and_Chladni_Patterns-A_Research_Review_of_Theory_Modelling_Simulation_and_Engineering_Applications

```
# - mostCommon : [63.486821]
# - leastCommon : [103.12671]
```

Figure 10.1: image("files/chladni01-c2ecf0ebfa159f82201f3e23aee08c3b.png", width: 90%)
 image("files/chladni02-bbb9873ae8d296a9aabb4264ac31e98e.png", width: 90%)

- 'chordSeq' / 'cs': Eine Akkordsequenz oder ein sich modulierendes Spektrum. Wie bei der einfachen seq wird auch hier angenommen, dass die zeitlichen Abstände zwischen den einzelnen Akkorden identisch sind. Grundeinstellung ist, dass die einzelnen Akkorde via `autoSortChords = True` vom tiefsten zum höchsten Ton sortiert werden.

```
from random import randint
from numpy import clip

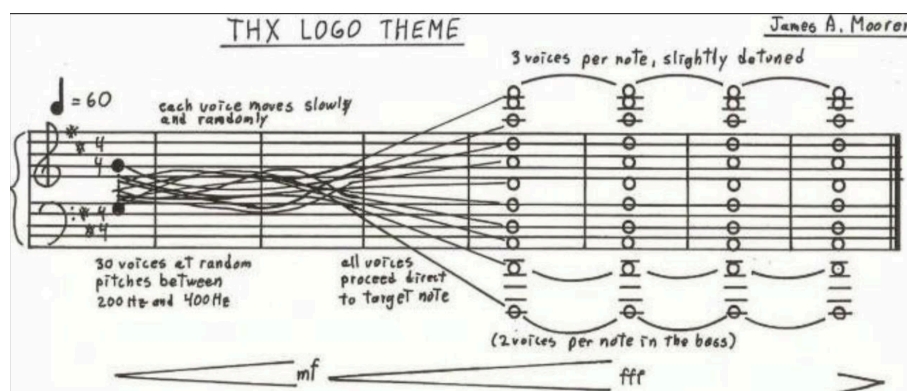
from GreasyPidgin.Phonon import Phonon
from GreasyPidgin.Pitch import spntof

stages = [[randint(200, 400) for _ in range(30)]]
for _ in range(8):
    newStage = [int(clip(f+randint(-20, 20), 200, 400)) for f in stages[-1]]
    stages.append(newStage)
finalChordSPN = [
    'd1', 'd1', 'd2', 'd2', 'd3', 'd3', 'd3',
    'a2', 'a2',
    'd4', 'd4', 'dtf4', 'dts4', 'd5', 'd5', 'dtf5', 'dts5', 'd6', 'd6', 'dtf6', 'dts6',
    'a4', 'atf4', 'ats4', 'a5', 'atf5', 'ats5',
    'fs5', 'fstf5', 'fst5']
stages.append([spntof(f) for f in finalChordSPN])

thxLogoTheme = Phonon(0, 29, stages, pitchUnit='hz', chordOrSeq='chordSeq')
print(thxLogoTheme)

# thxLogoTheme.parseMidi(noteSelectKey='chordSeq') # more on this later!

##### OUTPUT
#####
# Phonon(
#   id      : 0
#   startSec : 0.000, durSec: 29.000, endSec: 29.000
#   pitchesMidi :
#     - all      : [26.0, 38.0, ... ]
#     - mostCommon : [67.349958]
#     - leastCommon : [78.166667]
```



10.3 Export

Bisher ist, wie für das TemporalObject, nur der Export als MIDI-Datei möglich.

10.3.1 Midi

Mit der Methode `.parseMidi(folder, trackname, perNoteTracks, noteSelectKey)` kann ein Phonon als MIDI-Datei exportiert werden. Dabei gibt es verschiedenen Möglichkeiten, die Komplexität eines Phonon abzubilden. Als Standardeinstellung wird mit dem Parameter `noteSelectKey = 'lowest'` immer der tiefste vorhandene monophone Midiwert zum Export genutzt:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(0, 10, [['af4', 'a4'], ['af4', 'gqs4'], ['af4', 'a4']],
chordOrSeq='cs', pitchUnit='spn')
p.parseMidi('/tmp/default_is_')
print('\n\n')
p.parseMidi()

##### OUTPUT
#####
# Transformed pitch(es) to:
#   TemporalObject(id=0, start=0.000, dur=10.000, end=10.000, dyn=100.0 dB,
pitch=67.5, bpm=60.0, track=0)

# Midi file saved to: /tmp/default_is_lowest.mid
# Particle: 0 saved to /tmp/default_is_lowest.mid

# Transformed pitch(es) to:
#   TemporalObject(id=1, start=0.000, dur=10.000, end=10.000, dyn=100.0 dB,
pitch=67.5, bpm=60.0, track=0)

# Midi file saved to: /tmp/lowest.mid
# Particle: 0 saved to /tmp/lowest.mid
```

10.3.1.1 Optionen für noteSelectKey

Die weiteren Optionen sind: 'highest', 'mostCommon', 'leastCommon', 'all', 'allSet' / 'chord', 'seq' und 'chordSeq'.

- 'highest': Die höchste Note aus allen enthaltenen Midiwerten.
- 'mostCommon': Die häufigste Note aus allen enthaltenen Midiwerten.
- 'leastCommon': Die seltenste Note aus allen enthaltenen Midiwerten.
- 'all': Alle Noten als Blockakkord, inkl Dopplungen.
- 'allSet' / 'chord': Alle Noten als Blockakkord, aber ohne Dopplungen.
- 'seq': Ideal für homophone Tonhöhen-Bewegungen. Bei akkordischen Sequenzen wird Unterstimme ausgegeben. Alternativ kann beim initialisieren mit `autoSortChords = False` die erste erzeugte Sequenz exportiert werden.
- 'chordSeq': Alle Akkorde einer Sequenz, die Akkorde sind per se frei von Stimmkreuzungen. Um diese zu erlauben, muss beim initialisieren `autoSortChords = False` gesetzt werden.

Beispiel für eine Sequenz:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='s', pitchUnit='spn')

p.parseMidi(noteSelectKey='seq')
p.parseMidi(noteSelectKey='chordSeq')
p.parseMidi(noteSelectKey='lowest')
p.parseMidi(noteSelectKey='highest')
p.parseMidi(noteSelectKey='mostCommon')
p.parseMidi(noteSelectKey='leastCommon')
```



```
p.parseMidi(noteSelectKey='allSet')
p.parseMidi(noteSelectKey='chord')
```

Beispiel für Akkordsequenzen:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=[['af4', 'a4'], ['a4', 'gqs4'], ['af4', 'a4']],
    chordOrSeq='cs', pitchUnit='spn')

for noteSelectKey in
['seq', 'chordSeq', 'lowest', 'highest', 'mostCommon', 'leastCommon', 'allSet', 'chord']:
    p.parseMidi(noteSelectKey=noteSelectKey)
```

Beispiel für einen Akkord:

```
from GreasyPidgin.Phonon import Phonon
p = Phonon(
    0, 10,
    pitch=['af4', 'a4', 'a4', 'gqs4', 'af4', 'a4'],
    chordOrSeq='c', pitchUnit='spn')

for noteSelectKey in [
    'seq', 'chordSeq', 'lowest', 'highest',
    'mostCommon', 'leastCommon', 'allSet', 'chord'
]:
    p.parseMidi(noteSelectKey=noteSelectKey)
```